Lab01 Proposal: ArithLex, Arithmetic Input Scanner

1. Project title and members

Title: ArithLex, an arithmetic input scanner built with JFlex and Java.

Member	Student ID	Role	Responsibilities
Vuon Sethar Tykun	e20230389	A: Scanner author	arith.jflex macros and rules, token table
Vutha Daraleap	e20230820	B: Driver and Build	Main.java, generate/compile/run commands, README, coordination
Phy Sopanha	e20230530	C: Testing	Test inputs, expected outputs (written before running), results table, bug record
Sok Makara	e20230987	D: Documentation	Report, automaton drawing, slides, PDF copy, demo screenshots

All four members design the token rules together and review each other's work. Each member will explain one rule in the presentation: Vutha explains DECIMAL vs INTEGER, Sopanha explains GE vs GT, Makara explains the IDENTIFIER macro, and Sethar explains the error rule and position counting.

2. Purpose

ArithLex reads a UTF-8 text file containing simple arithmetic statements and prints every token with its type, lexeme, line and column (positions start at 1). It only recognizes tokens. It does not parse, evaluate or store anything.

3. Input format

The input is a text file with one or more statements, for example `total=12.5+qty*2;`.

Allowed characters: letters, digits, underscore, the decimal point inside a number, the symbols `= + - * / > ( ) ;`, and whitespace (space, tab, carriage return, newline). Any other character is reported as an error.

Decisions for edge cases:

- `-5` is MINUS followed by INTEGER (no negative number literals).
- A decimal needs digits on both sides of the point. `3.14` is one DECIMAL; a lone `.` is an error.
- `12abc` is INTEGER `12` followed by IDENTIFIER `abc`. Spacing and grammar are not checked at this stage.
- Whitespace is matched and skipped, and it still counts for column positions.
- There is no comment syntax. Any comment text such as `#` or `//` would be scanned as ordinary characters, so unsupported symbols are reported as errors.

- On an unexpected character, the scanner reports it with line and column and **continues** with the next character.
- At end of input the scanner stops cleanly and prints a summary of tokens and errors.

4. Planned token types (13)

Token	Pattern idea	Example
IDENTIFIER	letter or underscore, then letters, digits or underscores	total, qty_2
INTEGER	one or more digits	12
DECIMAL	digits, point, digits	12.5
ASSIGN	=	=
PLUS	+	+
MINUS	-	-
STAR	*	*
SLASH	/	/
GT	>	>
GE	>=	>=
LPAREN	(	(
RPAREN	)	)
SEMICOLON	;	;

Not counted in the 13: skipped whitespace and the ERROR report for unexpected characters.

5. Sample input and expected output

Input (first line of the file): `total=12.5+qty*2;`

Token type	Lexeme	Line	Column
IDENTIFIER	total	1	1
ASSIGN	=	1	6
DECIMAL	12.5	1	7
PLUS	+	1	11
IDENTIFIER	qty	1	12
STAR	*	1	15

Token type	Lexeme	Line	Column
INTEGER	2	1	16
SEMICOLON	;	1	17

End of input: 8 tokens, 0 errors.

6. Competing rules we plan to explain

1. **DECIMAL vs INTEGER:** for `12.5`, INTEGER matches `12` (2 characters) but DECIMAL matches `12.5` (4 characters). The longest match wins, so the result is one DECIMAL.
2. **GE vs GT:** for `>=`, GT matches `>` (1 character) but GE matches `>=` (2 characters). The longest match wins, so the result is one GE.

When two rules match the same length, the rule written first in the file wins, so the ERROR rule must be last.

7. Test plan

At least 15 distinct tests with expected output written before running: 6 normal, 5 boundary or competing-rule (including empty input, adjacent tokens such as `a+b`, and end of input right after a token), and 4 invalid (unsupported characters such as `@`, `#` and `$`, and a malformed number such as `3.` or `1.2.3`). We will also add a test for any bug we find and fix.

8. Feature left out of scope

Evaluating expressions. ArithLex lists tokens only and never calculates results. It also does not check grammar or matching parentheses, so a token sequence like `3 + ;` is accepted at the lexical level.